Refatoração para Testabilidade e Modelagem UML

1. Motivação

Na Fase 02, a `BatalhaService` concentrava quase toda a lógica e dependia de: `Random` (crítico, status, terreno, desempate), `Scanner` e `System.out` (entrada e saída), além de regras de tipo, terreno, status e item em `if/else` aninhados. Histórico e recompensa ficavam no `Main`. Isso tornava o projeto com alto acoplamento: dois testes usavam reflexão, dois precisavam de um `Random` anônimo, e não era possível testar uma batalha sem digitar no console.

2. Bugs corrigidos

ID	Problema	Correção
BUG-01	`Mochila.registrarUso` aceitava o 3º item sem exceção	Lança `IllegalStateException`
BUG-02	Envenenado com dano fixo	Dano progressivo por turno (`Status.danoNoTurno`)

Também foram removidos magic numbers e código morto, a ordem do fim de turno passou a ser status → terreno, e a `Mochila` ganhou uma cópia defensiva.

3. Classes refatoradas

Regras movidas para quem detém a informação (SRP, OCP): `TipoElemental` (matriz de vantagens), `Status` (dano, duração, expiração), `Terreno` (bônus de dano e cura) e `Itens` (condição de uso e efeito).

Responsabilidades extraídas da `BatalhaService` (SRP): `CalculadoraDano`, `Iniciativa`, `Historico`, `RecompensaService` e `ServicoConquistas`.

`BatalhaService` e `Main`: a service agora só orquestra os turnos, recebe tudo pelo construtor e devolve um `ResultadoBatalha`, salvo no repositório (se o repositório falhar, o erro é registrado e a batalha continua válida). O `Main` é o único lugar com `new Random()` e `Scanner`.

4. Acoplamento e Inversão de Dependência

As classes de regra passaram a depender de **interfaces definidas no próprio pacote `service`**, implementadas por `view` e `repository`:

Interface	Implementações	Substitui
`GeradorAleatorio`	`GeradorAleatorioPadrao`	`new Random()`
`EscolhaDeAcao`	`ConsoleEscolha`, `CpuEscolha`	`Scanner` e menu
`RegistroBatalha`	`ConsoleRegistro`, `RegistroSilencioso`	`System.out`
`RepositorioResultados`	`RepositorioResultadosMemoria`	Persistência direta
`FabricaDeBatalha`	`FabricaDeBatalhaTorneio`	`new BatalhaService` no `Torneio`
`AvisoTorneio`, `NotificadorConquista`	versões `Console...`	`println` nas regras

Consequências: cada colaborador vira `@Mock` no Mockito, permitindo controlar crítico e precisão, simular falha do repositório (`doThrow`) e validar a ordem dos eventos (`InOrder`); `ConsoleEscolha` e `CpuEscolha` são intercambiáveis (LSP), o que viabiliza batalhas CPU contra CPU no torneio; trocar repositório ou registro não exige alterar o service (OCP); interfaces pequenas (ISP).

Padrões de projeto usados: Strategy, Factory, Observer.